

COMPETENCY-BASED LEARNING MATERIALS (CBLM)

AI BASIC PYTHON DEVELOPER

TECHNICAL AND VOCATIONAL CERTIFICATE I — LEVEL II

Core Competency: Apply Structured Computer Programming

Module Title: Apply Structured Computer Programming

Learning Outcome: LO6 – Develop Program Using List

Nominal Duration: Theory 3.5 Hours / Practical 12 Hours

L.O 06: DEVELOP PROGRAM USING LIST

Upon completion of this learning outcome, students or trainees will have the following competencies:

1. Declare and initialize Python lists.
2. Explain the characteristics of Python lists.
3. Store multiple values in a list.
4. Access list elements using positive indexes.
5. Access list elements using negative indexes.
6. Retrieve a range of list elements using slicing.
7. Change the value of a specific list item.
8. Change a range of list values.
9. Determine the number of elements in a list using `len()`.
10. Add elements to lists.
11. Insert elements into specific positions.
12. Remove elements from lists.
13. Apply common Python list methods.
14. Sort and reverse list elements.
15. Copy lists appropriately.
16. Work with nested lists.
17. Apply lists to simple data-storage problems.
18. Test and debug Python list operations.

INFORMATION SHEET No. 6.1

Title: Understanding and Using Python Lists

Objective

Upon reading this Information Sheet, the trainee must be able to:

- Explain Python lists and their characteristics.
- Declare and initialize lists.
- Access list elements.
- Apply positive indexing.
- Apply negative indexing.
- Select a range of indexes using slicing.
- Modify list elements.
- Determine list length.
- Add and remove list elements.
- Apply common list methods.
- Explain list constructors.
- Work with nested lists.
- Explain basic differences between lists and other Python data structures.
- Apply lists for storing and processing multiple values.

1. Introduction to Python Lists

A Python list is a built-in data structure used to store multiple values in a single variable.

Instead of creating:

```
student1 = "Dara"  
student2 = "Sokha"  
student3 = "Vanna"
```

a list can be used:

```
students = ["Dara", "Sokha", "Vanna"]
```

The variable `students` contains three values.

Lists are one of the most commonly used data structures in Python.

2. Characteristics of Python Lists

Python lists have several important characteristics.

A list is:

- Ordered
- Changeable (mutable)
- Able to contain duplicate values
- Able to contain different data types
- Indexed
- Able to grow or shrink

Example:

```
data = ["Dara", 20, 3.5, True]
```

This list contains:

- String
- Integer
- Float
- Boolean

3. Declaring a List

Lists are normally declared using square brackets `[]`.

Syntax:

```
list_name = [item1, item2, item3]
```

Example:

```
fruits = ["apple", "banana", "orange"]
```

Another example:

```
numbers = [10, 20, 30, 40, 50]
```

4. Empty List

An empty list contains no elements.

Example:

```
students = []
```

Its length is:

```
print(len(students))
```

Output:

```
0
```

Values can be added later.

5. Creating Lists with list()

Python also provides the `list()` constructor.

Example:

```
fruits = list(("apple", "banana", "orange"))  
print(fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

6. Lists Containing Different Data Types

A list can contain values of different types.

Example:

```
student = ["Dara", 20, 85.5, True]  
print(student)
```

Output:

```
['Dara', 20, 85.5, True]
```

However, in many practical programs, lists contain related values of the same general type.

Example:

```
scores = [75, 80, 90, 65, 85]
```

7. List Indexing

Each element in a list has an index position.

Python starts indexing at:

0

Example:

```
fruits = ["apple", "banana", "orange", "mango"]
```

Index positions:

Item:	apple	banana	orange	mango
Index:	0	1	2	3

Therefore:

```
print(fruits[0])
```

Output:

apple

8. Accessing List Elements

Use the index number inside square brackets.

Example:

```
students = ["Dara", "Sokha", "Vanna"]
```

```
print(students[0])  
print(students[1])  
print(students[2])
```

Output:

Dara
Sokha
Vanna

9. Invalid List Index

Consider:

```
students = ["Dara", "Sokha", "Vanna"]
```

Valid positive indexes are:

0
1
2

Trying:

```
print(students[3])
```

causes:

IndexError

because index 3 does not exist.

10. Negative Indexing

Python supports negative indexes.

Negative indexing begins from the end of the list.

Example:

```
fruits = ["apple", "banana", "orange", "mango"]
```

Positions:

Item:	apple	banana	orange	mango
Positive:	0	1	2	3
Negative:	-4	-3	-2	-1

Therefore:

```
print(fruits[-1])
```

Output:

mango

And:

```
print(fruits[-2])
```

Output:

orange

11. List Slicing

List slicing retrieves a range of elements.

Syntax:

```
list[start:end]
```

The start index is included.

The end index is excluded.

Example:

```
numbers = [10, 20, 30, 40, 50]
```

```
print(numbers[1:4])
```

Output:

```
[20, 30, 40]
```

Indexes selected:

```
1  
2  
3
```

Index 4 is not included.

12. Slice from the Beginning

Example:

```
numbers = [10, 20, 30, 40, 50]  
  
print(numbers[:3])
```

Output:

```
[10, 20, 30]
```

Python starts from the beginning when the first index is omitted.

13. Slice to the End

Example:

```
numbers = [10, 20, 30, 40, 50]  
  
print(numbers[2:])
```

Output:

```
[30, 40, 50]
```

Python continues to the end when the ending index is omitted.

14. Negative Index Slicing

Negative indexes can also be used with slicing.

Example:

```
fruits = ["apple", "banana", "orange", "mango", "grape"]  
  
print(fruits[-4:-1])
```

Output:

```
['banana', 'orange', 'mango']
```

15. Changing a Specific List Item

Python lists are mutable.

This means their contents can be changed after creation.

Example:

```
fruits = ["apple", "banana", "orange"]  
  
fruits[1] = "mango"  
  
print(fruits)
```

Output:

```
['apple', 'mango', 'orange']
```

The item at index 1 changed from:

banana

to:

mango

16. Changing Multiple List Items

A range of items can also be replaced.

Example:

```
fruits = ["apple", "banana", "orange", "mango"]  
  
fruits[1:3] = ["grape", "watermelon"]  
  
print(fruits)
```

Output:

```
['apple', 'grape', 'watermelon', 'mango']
```

17. Determining List Length

The built-in `len()` function determines the number of elements in a list.

Example:

```
students = ["Dara", "Sokha", "Vanna", "Malis"]  
  
print(len(students))
```

Output:

4

The list contains four elements.

18. Adding Items to a List

Python provides several methods for adding items.

Common methods include:

```
append()  
insert()  
extend()
```

19. append() Method

`append()` adds one item to the end of a list.

Example:

```
fruits = ["apple", "banana"]  
  
fruits.append("orange")  
  
print(fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

20. insert() Method

`insert()` adds an item at a specified index.

Syntax:

```
list.insert(index, value)
```

Example:

```
fruits = ["apple", "orange"]  
  
fruits.insert(1, "banana")  
  
print(fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

21. extend() Method

`extend()` adds elements from another iterable to the list.

Example:

```
fruits = ["apple", "banana"]  
more_fruits = ["orange", "mango"]
```

```
fruits.extend(more_fruits)

print(fruits)
```

Output:

```
['apple', 'banana', 'orange', 'mango']
```

22. Removing Items from Lists

Common methods include:

```
remove()
pop()
clear()
```

The `del` keyword can also remove list elements.

23. remove() Method

`remove()` removes a specified value.

Example:

```
fruits = ["apple", "banana", "orange"]

fruits.remove("banana")

print(fruits)
```

Output:

```
['apple', 'orange']
```

24. pop() Method

`pop()` removes an element by index and returns the removed value.

Example:

```
fruits = ["apple", "banana", "orange"]

removed_item = fruits.pop(1)

print(fruits)
print(removed_item)
```

Output:

```
['apple', 'orange']
banana
```

Without an index:

```
fruits.pop()
```

removes the last element.

25. del Keyword

The del keyword can remove an element by index.

Example:

```
fruits = ["apple", "banana", "orange"]  
  
del fruits[1]  
  
print(fruits)
```

Output:

```
['apple', 'orange']
```

26. clear() Method

clear() removes all elements from the list.

Example:

```
fruits = ["apple", "banana", "orange"]  
  
fruits.clear()  
  
print(fruits)
```

Output:

```
[]
```

The list still exists but is empty.

27. sort() Method

sort() sorts list elements.

Example:

```
fruits = ["mango", "apple", "orange", "banana"]  
  
fruits.sort()  
  
print(fruits)
```

Output:

```
['apple', 'banana', 'mango', 'orange']
```

28. Sorting Numbers

Example:

```
numbers = [50, 10, 40, 20, 30]

numbers.sort()

print(numbers)
```

Output:

```
[10, 20, 30, 40, 50]
```

29. Descending Sort

Use:

```
reverse=True
```

Example:

```
numbers = [50, 10, 40, 20, 30]

numbers.sort(reverse=True)

print(numbers)
```

Output:

```
[50, 40, 30, 20, 10]
```

30. reverse() Method

reverse() reverses the current order of the list.

Example:

```
numbers = [10, 20, 30, 40]

numbers.reverse()

print(numbers)
```

Output:

```
[40, 30, 20, 10]
```

reverse() does not perform numerical or alphabetical sorting. It simply reverses the existing order.

31. count() Method

count() determines how many times a specified value appears.

Example:

```
numbers = [10, 20, 10, 30, 10]

print(numbers.count(10))
```

Output:

3

32. index() Method

index() returns the index of the first occurrence of a specified value.

Example:

```
fruits = ["apple", "banana", "orange"]  
  
print(fruits.index("banana"))
```

Output:

1

33. copy() Method

copy() creates a shallow copy of a list.

Example:

```
fruits = ["apple", "banana", "orange"]  
  
new_fruits = fruits.copy()  
  
print(new_fruits)
```

Output:

```
['apple', 'banana', 'orange']
```

This is preferable to simply writing:

```
new_fruits = fruits
```

when an independent top-level list copy is required.

34. Common List Methods

Method	Purpose
append()	Add item to end
insert()	Add item at specified position
extend()	Add multiple elements
remove()	Remove specified value
pop()	Remove and return item
clear()	Remove all items
sort()	Sort list
reverse()	Reverse list order

Method	Purpose
count()	Count occurrences
index()	Find position
copy()	Copy list

35. Checking Whether an Item Exists

The `in` operator can check whether an item exists in a list.

Example:

```
fruits = ["apple", "banana", "orange"]

if "banana" in fruits:
    print("Banana exists in the list")
```

Output:

```
Banana exists in the list
```

36. Nested Lists

A list can contain another list.

Example:

```
students = [
    ["Dara", 85],
    ["Sokha", 90],
    ["Vanna", 75]
]
```

This is called a nested list.

37. Accessing Nested List Elements

Consider:

```
students = [
    ["Dara", 85],
    ["Sokha", 90],
    ["Vanna", 75]
]
```

To display:

```
Sokha
```

use:

```
print(students[1][0])
```

To display Sokha's score:

```
print(students[1][1])
```

Output:

90

38. Lists Compared with Other Data Structures

Python provides several built-in collection types.

Data Structure	Ordered	Changeable	Duplicates
List	Yes	Yes	Yes
Tuple	Yes	No	Yes
Set	No fixed positional indexing	Yes	No
Dictionary	Key-based/insertion ordered	Yes	Keys: No

Lists are especially useful when:

- Order matters.
- Values may change.
- Duplicate values are acceptable.
- Index-based access is required.

39. Practical Example — Student Scores

```
scores = [80, 75, 90, 85]

print("Scores:", scores)
print("First score:", scores[0])
print("Last score:", scores[-1])
print("Number of scores:", len(scores))
```

Output:

```
Scores: [80, 75, 90, 85]
First score: 80
Last score: 85
Number of scores: 4
```

40. Practical Example — Modify Student Score

```
scores = [80, 75, 90]

scores[1] = 85

print(scores)
```

Output:

```
[80, 85, 90]
```

41. Practical Example — Add and Remove Students

```
students = ["Dara", "Sokha", "Vanna"]
```

```
students.append("Malis")
students.remove("Sokha")

print(students)
```

Output:

```
['Dara', 'Vanna', 'Malis']
```

42. Practical Example — Sort Scores

```
scores = [75, 90, 60, 85, 70]

scores.sort()

print(scores)
```

Output:

```
[60, 70, 75, 85, 90]
```

43. List Operation Process

A typical list program follows:

```
Analyze Requirement
      ↓
Declare List
      ↓
Store Elements
      ↓
Access Elements
      ↓
Modify Elements
      ↓
Add/Remove Elements
      ↓
Apply List Methods
      ↓
Display Results
      ↓
Test Program
      ↓
Verify Output
```

44. Common List Errors

Invalid Index

Incorrect:

```
students = ["Dara", "Sokha"]

print(students[5])
```

This causes:

```
IndexError
```

Incorrect Method Name

Incorrect:

```
students.add("Vanna")
```

For a list, use:

```
students.append("Vanna")
```

Missing Parentheses

Incorrect:

```
students.append "Vanna"
```

Correct:

```
students.append("Vanna")
```

Removing a Value That Does Not Exist

Example:

```
students = ["Dara", "Sokha"]
```

```
students.remove("Vanna")
```

This causes:

```
ValueError
```

because "Vanna" is not in the list.

45. Debugging List Programs

When a list program produces an error:

1. Check the list declaration.
2. Check square brackets.
3. Check commas between elements.
4. Check index positions.
5. Check negative indexes.
6. Check slicing ranges.
7. Check method names.
8. Check method arguments.
9. Check whether the item exists before removing it.
10. Check expected and actual list contents.
11. Run the program again.

12. Verify the final output.

SELF-CHECK No. 06

Part A — Multiple Choice

1. Which brackets declare a Python list?

- A. () B. {} C. [] D. <>

2. What is the first list index?

- A. 1 B. 0 C. -1 D. 2

3. What does index -1 represent?

- A. First element B. Second element C. Last element D. Invalid element

4. Which function determines list length?

- A. size() B. length() C. len() D. countall()

5. Which method adds an item to the end?

- A. add() B. append() C. insertend() D. push()

6. Which method inserts an item at a specific position?

- A. insert() B. append() C. add() D. place()

7. Which method removes a specified value?

- A. delete() B. remove() C. clear() D. erase()

8. Which method removes an item by index and returns it?

- A. pop() B. clear() C. remove() D. delete()

9. Which method sorts a list?

- A. order() B. arrange() C. sort() D. sortedlist()

10. Which method removes all list elements?

- A. delete() B. remove() C. clear() D. pop()

11. What does fruits[1:3] select?

- A. Indexes 1, 2 and 3 B. Indexes 1 and 2 C. Only index 3 D. Indexes 0 to 3

12. Which method reverses the current list order?

- A. reverse() B. back() C. flip() D. sort()

13. Which method counts occurrences of a value?

- A. len() B. count() C. total() D. number()

14. Which method creates a shallow copy of a list?

- A. clone() B. duplicate() C. copy() D. new()

15. Python lists are:

- A. Immutable B. Mutable C. Always numeric D. Unable to contain duplicates

ANSWER KEY No. 06

Part A

No.	Answer
1	C
2	B
3	C
4	C
5	B
6	A
7	B
8	A
9	C
10	C
11	B
12	A
13	B
14	C
15	B

OPERATION SHEET No. 06

Title: Create, Access, and Manipulate Python Lists

Performance Objective

Given a computer with Python installed and an approved Python development environment, the trainee must create a list, access its elements, apply positive and negative indexing, retrieve ranges, modify values, determine list length and apply common list methods.

Description

This operation demonstrates the standard procedure for creating and manipulating Python lists.

Picture/Overview

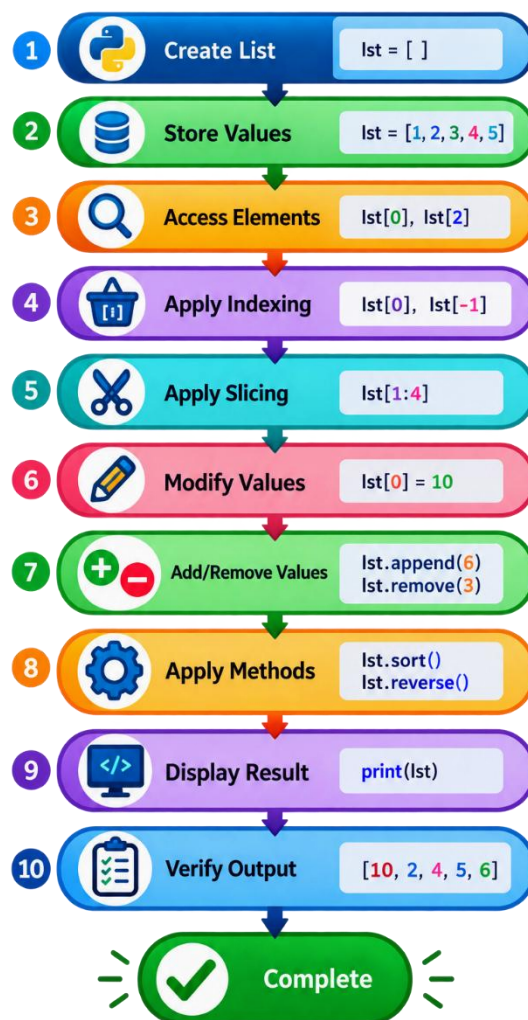


Figure 1:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/VS Code/PyCharm	Software	1

No.	Description	Unit	Qty
5	Command Prompt/Terminal	Software	1
6	Python Programming Reference Guide	Copy	1
7	CBLM	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Open Python IDE	Development environment opens correctly
2	Create a new .py file	Source file is ready
3	Declare a list	Correct [] syntax is used
4	Store multiple values	Values are correctly separated
5	Access positive index	Correct element is returned
6	Access negative index	Correct element from end is returned
7	Apply slicing	Required range is returned
8	Change a list item	Correct element is modified
9	Determine list length	len() returns correct result
10	Add list item	append()/insert() works correctly
11	Remove list item	Appropriate removal method is used
12	Sort list	Elements are correctly arranged
13	Save program	.py extension is used
14	Execute program	Program runs successfully
15	Verify results	Output matches requirements

Operation Procedure

Create:

```
fruits = ["apple", "banana", "orange", "mango"]
```

Access:

```
print(fruits[0])
print(fruits[-1])
```

Slice:

```
print(fruits[1:3])
```

Modify:

```
fruits[1] = "grape"
```

Determine length:

```
print(len(fruits))
```

Add:

```
fruits.append("watermelon")
```

Remove:

```
fruits.remove("orange")
```

Sort:

```
fruits.sort()
```

Display:

```
print(fruits)
```

Save as:

```
list_operations.py
```

Run the program and verify all results.

Assessment Method

- Direct observation
- Practical demonstration
- Source-code inspection
- Program execution
- Output verification

TASK SHEET No. 6.1

Title: Declare and Access Python Lists

Performance Task

Create Python lists and access their elements using positive and negative indexing.

Performance Objective

Given a Python development environment, the trainee must declare lists correctly and retrieve specified elements using positive and negative index positions.

Description

The trainee practices the fundamental operations required to create and access list data.

Picture/Overview

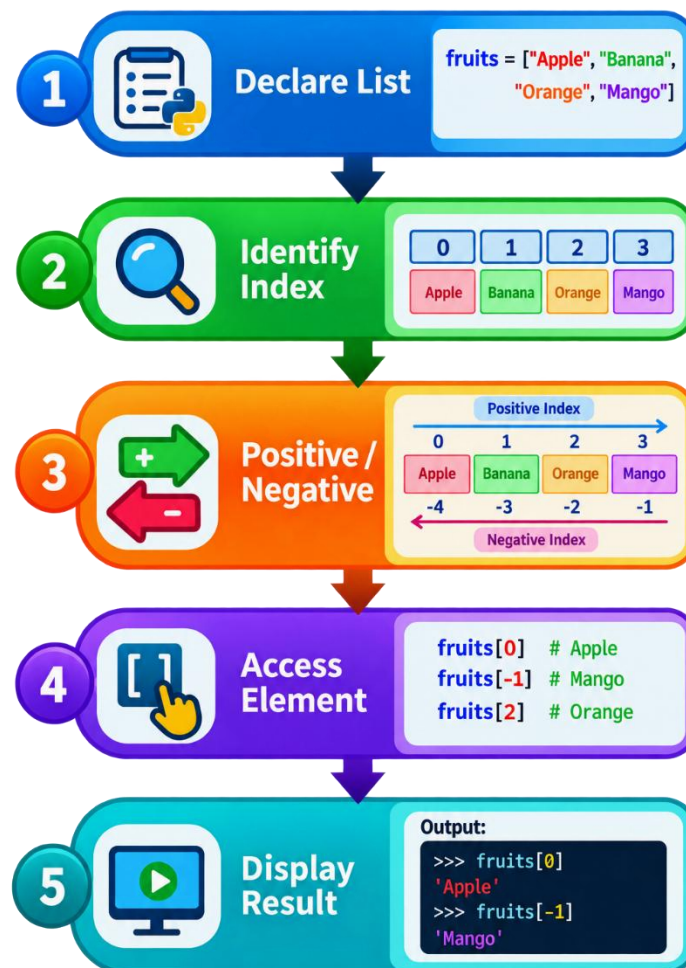


Figure 2:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Programming Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Declare list	Correct square brackets are used
2	Store five elements	Required values are stored
3	Access index 0	First element is displayed
4	Access index 2	Third element is displayed
5	Access index -1	Last element is displayed
6	Access index -2	Second-last element is displayed
7	Verify results	Correct elements are returned

Practical Exercise

Create:

```
students = ["Dara", "Sokha", "Vanna", "Malis", "Ratha"]
```

Display:

```
print(students[0])
print(students[2])
print(students[-1])
print(students[-2])
```

Record:

Expression	Expected Result	Actual Result
students[0]	Dara	_____
students[2]	Vanna	_____
students[-1]	Ratha	_____
students[-2]	Malis	_____

Assessment Method

- Direct observation
- Practical exercise
- Source-code inspection
- Output verification

TASK SHEET No. 6.2

Title: Apply List Slicing and Modify List Elements

Performance Task

Retrieve specified ranges of list elements and modify individual or multiple values.

Performance Objective

Given a Python list, the trainee must correctly apply slicing and modify specified elements according to requirements.

Description

This activity develops skills in index ranges and list modification.

Picture/Overview



Figure 3:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1

No.	Description	Unit	Qty
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create list	Required values are stored
2	Slice indexes 1:4	Correct range is returned
3	Slice from beginning	Correct elements are returned
4	Slice to end	Correct elements are returned
5	Change specific item	Correct index is modified
6	Change range	Required values are updated
7	Verify list	Final contents are correct

Practical Exercise

Create:

```
numbers = [10, 20, 30, 40, 50, 60]
```

Execute:

```
print(numbers[1:4])
print(numbers[:3])
print(numbers[3:])
```

Change:

```
numbers[2] = 35
```

Then:

```
numbers[3:5] = [45, 55]
```

Display:

```
print(numbers)
```

Record the final list: _____

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

TASK SHEET No. 6.3

Title: Determine List Length and Add List Elements

Performance Task

Determine the number of elements in a list and add new elements using appropriate list methods.

Performance Objective

Given an existing list, the trainee must use `len()`, `append()`, `insert()`, and `extend()` correctly.

Description

The trainee practices common list growth operations.

Picture/Overview

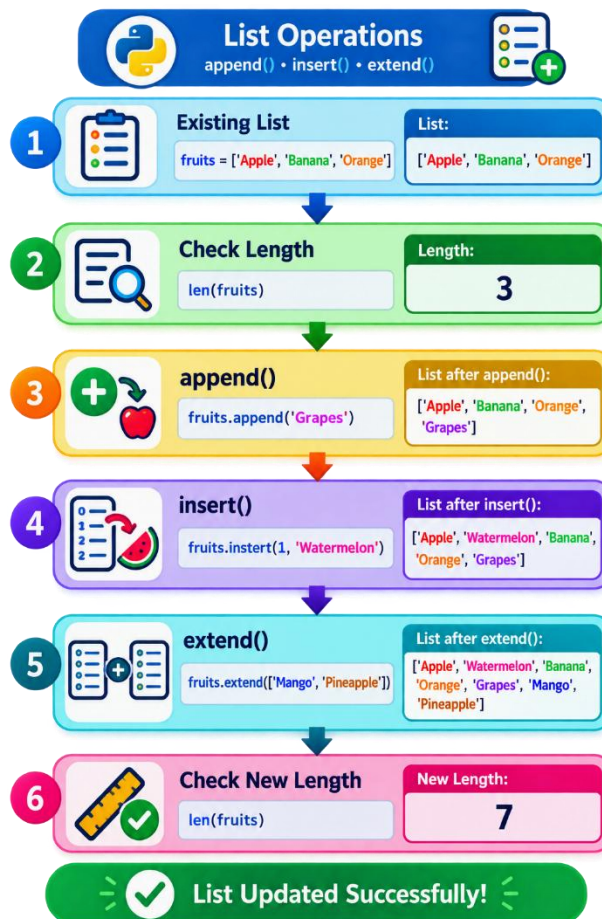


Figure 4:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create list	List is correctly declared
2	Apply len()	Correct length is displayed
3	Apply append()	Item is added at end
4	Apply insert()	Item is added at specified position
5	Apply extend()	Multiple elements are added
6	Apply len() again	Updated length is correct
7	Verify final list	Contents match requirements

Practical Exercise

```
students = ["Dara", "Sokha", "Vanna"]
```

```
print("Original length:", len(students))
```

```
students.append("Malis")  
students.insert(1, "Ratha")  
students.extend(["Sophy", "Nita"])
```

```
print(students)  
print("New length:", len(students))
```

Record:

Original length: _____

Final list: _____

New length: _____

Assessment Method

- Direct observation
- Practical exercise
- Source-code review
- Output verification

TASK SHEET No. 6.4

Title: Remove and Organize List Elements

Performance Task

Remove, sort, reverse and inspect list elements using built-in list methods.

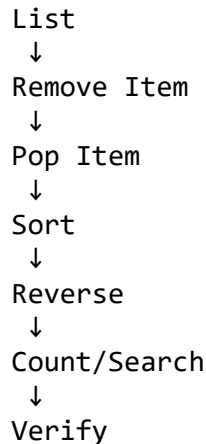
Performance Objective

Given a Python list, the trainee must correctly apply `remove()`, `pop()`, `sort()`, `reverse()`, `count()`, and `index()`.

Description

This activity develops competency in common list management methods.

Picture/Overview



Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create list	Required values are stored
2	Apply <code>remove()</code>	Correct value is removed
3	Apply <code>pop()</code>	Correct indexed item is removed
4	Apply <code>sort()</code>	Values are correctly sorted
5	Apply <code>reverse()</code>	Current order is reversed
6	Apply <code>count()</code>	Correct occurrence count is returned
7	Apply <code>index()</code>	Correct index is returned
8	Verify results	Output is accurate

Practical Exercise

Create:

```
scores = [80, 70, 90, 60, 70, 85]
```

Perform:

```

scores.remove(60)
print(scores)

removed_score = scores.pop(0)
print("Removed:", removed_score)

scores.sort()
print("Sorted:", scores)

scores.reverse()
print("Reversed:", scores)

print("Count of 70:", scores.count(70))
print("Index of 90:", scores.index(90))

```

Record your outputs:

After remove: _____

Removed score: _____

Sorted list: _____

Reversed list: _____

Count of 70: _____

Index of 90: _____

Assessment Method

- Direct observation
- Practical demonstration
- Program execution
- Output verification

TASK SHEET No. 6.5

Title: Create and Access Nested Lists

Performance Task

Create a nested list for storing student names and scores and retrieve specified information.

Performance Objective

Given student data, the trainee must organize multiple values using nested lists and access the required elements using multiple indexes.

Description

The trainee applies lists to structured data storage.

Picture/Overview

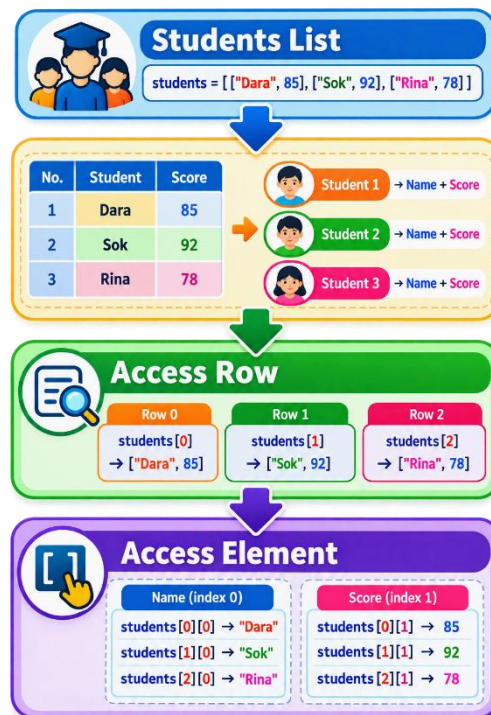


Figure 5:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Computer/Laptop	Unit	1
2	Python 3.x	Copy	1
3	Python IDE	Software	1
4	Worksheet	Copy	1

Supplier

Training institution.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Create outer list	Main list is correct
2	Create inner lists	Student data is correctly grouped
3	Access first student	Correct nested indexes are used

No.	Procedure/Step	Key Performance
4	Access second score	Correct value is returned
5	Modify a nested value	Correct element is updated
6	Display final data	Updated structure is correct

Practical Exercise

Create:

```
students = [
    ["Dara", 85],
    ["Sokha", 90],
    ["Vanna", 75]
]
```

Display Dara's name:

```
print(students[0][0])
```

Display Sokha's score:

```
print(students[1][1])
```

Change Vanna's score to 80:

```
students[2][1] = 80
```

Display:

```
print(students)
```

Assessment Method

- Direct observation
- Practical exercise
- Program execution
- Output verification

JOB SHEET No. 6.1

Title: Develop a Student Score Management Program Using Lists

Performance Task

Develop a complete Python program that uses lists to store and manage student scores.

Performance Objective

Given a computer, Python IDE and specified program requirements, the trainee must create lists, access values using positive and negative indexes, retrieve ranges, update elements, determine list length, add and remove values, sort data, use common list methods, test the program and verify the output.

Description

Develop:

`student_score_manager.py`

The program will demonstrate the major list operations required in LO6.

Picture/Overview



Figure 6:

Tools, Equipment and Materials

No.	Description	Unit	Qty
1	Desktop Computer/Laptop	Unit	1
2	Keyboard and Mouse	Set	1
3	Python 3.x	Copy	1
4	Python IDLE/VS Code/PyCharm	Software	1
5	Programming Worksheet	Copy	1
6	Python Reference Guide	Copy	1
7	CBLM	Copy	1

Supplier

Training institution / authorized software provider.

Procedure/Step | Key Performance

No.	Procedure/Step	Key Performance
1	Analyze program requirements	Required list operations are identified
2	Open Python IDE	Environment opens correctly
3	Create program file	New source file is ready
4	Declare score list	Correct list syntax is used
5	Access first element	Positive index works correctly
6	Access last element	Negative index works correctly
7	Retrieve score range	Correct slicing syntax is used
8	Change specific score	Required value is updated
9	Determine list length	len() produces correct result
10	Append score	Value is added to end
11	Insert score	Value is added at specified position
12	Remove score	Required value is removed
13	Pop score	Correct element is removed
14	Sort scores	List is correctly arranged
15	Count value	Correct occurrence count is displayed
16	Find index	Correct index is returned
17	Display final list	Final data is accurate
18	Save program	.py extension is used
19	Execute program	Program runs successfully
20	Test operations	All required operations are tested
21	Debug errors	Errors are corrected
22	Verify output	Output matches requirements

Program Requirements

Create a Python program that:

1. Declares a list containing at least five student scores.
2. Displays the complete list.
3. Displays the first score.
4. Displays the last score using negative indexing.
5. Displays scores from a specified index range.
6. Changes one score.
7. Displays the number of scores.
8. Appends a new score.

9. Inserts a score at a specified position.
10. Removes a specified score.
11. Uses pop().
12. Sorts scores.
13. Displays the highest and lowest values after sorting.
14. Counts occurrences of a specified score.
15. Displays the final list.

Example Program

Student Score Management Program

```
scores = [75, 85, 90, 65, 80]
```

```
print("STUDENT SCORE MANAGEMENT")
print("Original scores:", scores)
```

Access elements

```
print("First score:", scores[0])
print("Last score:", scores[-1])
```

List slicing

```
print("Scores from index 1 to 3:", scores[1:4])
```

Change a score

```
scores[3] = 70
print("After update:", scores)
```

Determine List Length

```
print("Number of scores:", len(scores))
```

Add elements

```
scores.append(95)
print("After append:", scores)
```

```
scores.insert(2, 88)
print("After insert:", scores)
```

Remove element

```
scores.remove(75)
print("After remove:", scores)
```

Pop element

```
removed_score = scores.pop()
print("Popped score:", removed_score)
```

Sort

```
scores.sort()
print("Sorted scores:", scores)
```

Lowest and highest after sorting

```
print("Lowest score:", scores[0])
print("Highest score:", scores[-1])
```

```
# Count
print("Number of 85 scores:", scores.count(85))

# Find index
print("Index of 90:", scores.index(90))

print("Final scores:", scores)
```

Expected Main Results

Initial:

[75, 85, 90, 65, 80]

After changing:

[75, 85, 90, 70, 80]

After append:

[75, 85, 90, 70, 80, 95]

After insert:

[75, 85, 88, 90, 70, 80, 95]

After removing 75:

[85, 88, 90, 70, 80, 95]

After pop():

[85, 88, 90, 70, 80]

Sorted:

[70, 80, 85, 88, 90]

Lowest:

70

Highest:

90

WORKSHEET No. 6.1

Title: Python List Practice and Verification Worksheet

Trainee Name: _____

Date: _____

Part A — List Declaration

Write a list containing five programming languages.

languages = _____

Part B — Indexing

Given:

```
fruits = ["apple", "banana", "orange", "mango", "grape"]
```

Complete:

Expression	Result
fruits[0]	_____
fruits[2]	_____
fruits[-1]	_____
fruits[-2]	_____

Part C — Slicing

Determine the result:

```
numbers = [10, 20, 30, 40, 50, 60]
```

Expression	Result
numbers[1:4]	_____
numbers[:3]	_____
numbers[3:]	_____
numbers[-3:]	_____

Part D — Modify List

Given:

```
colors = ["red", "green", "blue"]
```

Write code to change "green" to "yellow".

Part E — List Length

Given:

```
scores = [80, 75, 90, 85, 70]
```

Write code to display the number of scores.

Expected result:

Part F — Add Items

Complete:

```
students = ["Dara", "Sokha"]
```

```
students._____("Vanna")
```

```
students._____(1, "Malis")
```

Part G — Remove Items

Complete:

```
students = ["Dara", "Sokha", "Vanna"]
```

```
students._____("Sokha")
```

To remove the final item:

```
students.______()
```

Part H — Sort List

Complete:

```
numbers = [50, 10, 30, 20, 40]
```

```
numbers.______()
```

```
print(numbers)
```

Expected:

Part I — Nested List

Given:

```
students = [  
    ["Dara", 80],  
    ["Sokha", 90],  
    ["Vanna", 75]  
]
```

Write code to display "Sokha":

Write code to display 75:

Part J — Debugging

Correct:

```
fruits = ["apple", "banana", "orange"]
```

```
print(fruits[3])
```

Correct code:

Explain the errors:

PERFORMANCE CHECKLIST No. 6.1

Title: Develop Program Using List

Name: _____

Date: _____

Did you use all these performance criteria during your task?

No.	Criteria	Yes	No
1	Declared a Python list correctly	☐	☐
2	Initialized list with appropriate values	☐	☐
3	Identified basic characteristics of lists	☐	☐
4	Stored multiple values in a list	☐	☐
5	Accessed first list element	☐	☐
6	Accessed elements using positive indexing	☐	☐
7	Accessed last element using negative indexing	☐	☐
8	Applied other negative indexes correctly	☐	☐
9	Specified a range of indexes	☐	☐
10	Applied list slicing correctly	☐	☐
11	Applied slicing from the beginning	☐	☐
12	Applied slicing to the end	☐	☐
13	Changed the value of a specific list item	☐	☐
14	Changed a range of list items	☐	☐
15	Determined list length using len()	☐	☐
16	Applied append() correctly	☐	☐
17	Applied insert() correctly	☐	☐
18	Applied extend() correctly	☐	☐
19	Applied remove() correctly	☐	☐
20	Applied pop() correctly	☐	☐
21	Applied clear() correctly	☐	☐
22	Applied sort() correctly	☐	☐
23	Applied descending sorting where required	☐	☐
24	Applied reverse() correctly	☐	☐
25	Applied count() correctly	☐	☐
26	Applied index() correctly	☐	☐
27	Applied copy() appropriately	☐	☐
28	Used in to check list membership	☐	☐
29	Created a nested list	☐	☐

No.	Criteria	Yes	No
30	Accessed nested list elements	☐	☐
31	Modified nested list data	☐	☐
32	Selected appropriate list methods	☐	☐
33	Saved the program with .py extension	☐	☐
34	Executed the program successfully	☐	☐
35	Tested list operations	☐	☐
36	Identified invalid indexes	☐	☐
37	Identified incorrect list methods	☐	☐
38	Corrected syntax/runtime errors	☐	☐
39	Verified final list contents	☐	☐
40	Verified program output	☐	☐

Comment:

Overall Assessment

Competent: ☐ Yes ☐ No

Needs Further Practice: ☐ Yes ☐ No

Name and Signature of Trainer

Name: _____

Signature: _____

Date: _____